claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a90498ab003c8ee72.jsonl`
- SHA-256: `597993aaea38239aa5b2833d090425087bf19f8b4a3d8bd09c2db62b05cc415d`
- Source modified: `2026-05-31T04:19:06+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:18:31.639Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/bank-statement?view=doc-intel-4.0.0
Title: Azure prebuilt bank statement (US) extraction model ? Document Intelligence (Microsoft Learn)
Found via: Approval-gated paid/cloud fallbacks & data terms (cost/tradeoffs) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
   - be a concrete, checkable statement (not vague generalities)
   - include a direct quote from the source as support
   - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:34.046Z)

I'll fetch the Azure bank statement documentation page and extract the relevant claims.

3. user (2026-05-31T04:18:39.950Z)

---
layout: Conceptual
monikers:
- doc-intel-4.0.0
defaultMoniker: doc-intel-4.0.0
versioningType: Ranged
title: Bank statement US extraction model - Document Intelligence - Foundry Tools | Microsoft Learn
canonicalUrl: https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/bank-statement?view=doc-intel-4.0.0
config_moniker_range: '>=doc-intel-2.1.0'
breadcrumb_path: ../../../breadcrumb/azure-ai/toc.json
feedback_help_link_url: https://learn.microsoft.com/answers/tags/440/document-intelligence/
feedback_help_link_type: get-help-at-qna
feedback_product_url: https://feedback.azure.com/d365community/forum/09041fae-0b25-ec11-b6e6-000d3a4f0858?c=7a8853b4-0b25-ec11-b6e6-000d3a4f0858

```
feedback_system: Standard
permissioned-type: public
recommendations: true
recommendation_types:
- Training
- Certification
uhfHeaderId: azure-ai-foundry
ms.suite: office
author: laujan
learn_banner_products:
- azure
manager: nitinme
ms.author: lajanuar
ms.collection: ce-skilling-ai-copilot
ms.update-cycle: 365-days
ms.service: azure-ai-document-intelligence
description: OCR and machine learning based bank statement US extraction in Document
Intelligence extracts key data from bank statements.
ms.topic: concept-article
ms.date: 2025-11-18T00:00:00.0000000Z
ms.custom: references_regions
locale: en-us
document_id: 282e72f8-c585-a82f-2342-54a414b3b286
document_version_independent_id: 2b4cd732-e6d7-55b7-8233-bebb63251303
updated_at: 2026-01-23T23:20:00.0000000Z
original_content_git_url: https://github.com/MicrosoftDocs/azure-ai-docs-
pr/blob/live/articles/ai-services/document-intelligence/prebuilt/bank-statement.md
gitcommit: https://github.com/MicrosoftDocs/azure-ai-docs-
pr/blob/3e4fe1f6f46e16f49b74629bdb309c1137e331d7/articles/ai-services/document-
intelligence/prebuilt/bank-statement.md
git_commit_id: 3e4fe1f6f46e16f49b74629bdb309c1137e331d7
default_moniker: doc-intel-4.0.0
site_name: Docs
depot_name: Learn.azure-ai
page_type: conceptual
toc_rel: ../toc.json
word_count: 565
asset_id: ai-services/document-intelligence/prebuilt/bank-statement
moniker_range_name: 4ed3e8b5b892cc58bc8f80a438374376
monikers:
- doc-intel-4.0.0
item_type: Content
source_path: articles/ai-services/document-intelligence/prebuilt/bank-statement.md
cmProducts:
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/91df3d29-834f-4393-92e2-5e393da1897c
spProducts:
- https://microsoft-
devrel.poolparty.biz/DevRelOfferingOntology/805a3516-ebe7-4153-baf7-2b9dfcf7b866
platformId: da3dea99-b478-c3a1-7385-4faa5752a4cf
---
```


Bank statement US extraction model - Document Intelligence - Foundry Tools | Micros

The Document Intelligence bank statement model combines powerful Optical Character Recognition (OCR) capabilities with deep learning models to analyze and extract data from US bank statements. The API analyzes printed bank statements; extracts key information such as account number, bank details, statement details, transaction details, and fees; and returns a structured JSON data representation. With V4.0 GA, you can now extract check tables in the US bank statements.

Feature	version	Model ID
Bank statement model	v4.0: [**2024-11-30**](/en-us/rest/api/aiservices/operation-

groups?view=rest-aiservices-v4.0%20%282024-11-30%29&preserve-view=true) (GA) | **`prebuilt-bankStatement.us`** |

Bank statement data extraction

A bank statement helps review account's activities during a specified period. It's an official statement that helps in detecting fraud, tracking expenses, accounting errors and record the period's activities. See how data is extracted using the `prebuilt-bankStatement.us` model. You need the following resources:

- An Azure subscription?you can [create one for free](https://azure.microsoft.com/pricing/purchase-options/azure-account?cid=msft_learn)
- A [Document Intelligence instance](https://portal.azure.com/#create/Microsoft.CognitiveServicesFormRecognizer) in the Azure portal. You can use the free pricing tier (`F0`) to try the service. After your resource deploys, select **Go to resource** to get your key and endpoint.

    ![Screenshot of keys and endpoint location in the Azure portal.](../media/containers/keys-and-endpoint.png)

Document Intelligence Studio

1. On the [Document Intelligence Studio home page](https://documentintelligence.ai.azure.com/studio), select **bank statements**.
2. You can analyze the sample bank statement or upload your own files.
3. Select the **Run analysis** button and, if necessary, configure the **Analyze options** :

    ![Screenshot of Run analysis and Analyze options buttons in the Document Intelligence Studio.](../media/studio/run-analysis-analyze-options.png)

Input requirements

The following file formats are supported.

Model	PDF	Image:JPEG/JPG, PNG, BMP, TIFF, HEIF	Office:Word (DOCX), Excel (XLSX), PowerPoint (PPTX), HTML
Read	?	?	?
Layout	?	?	?
General document	?	?	
Prebuilt	?	?	
Custom extraction	?	?	
Custom classification	?	?	?

- **Photos and scans**: For best results, provide one clear photo or high-quality scan per document.
- **PDFs and TIFFs**: For PDFs and TIFFs, up to 2,000 pages can be processed. (With a free-tier subscription, only the first two pages are processed.)
- **File size**: The file size for analyzing documents is 500 MB for the paid (S0) tier and 4 MB for the free (F0) tier.
- **Image dimensions**: The dimensions must be between 50 pixels x 50 pixels and 10,000 pixels x 10,000 pixels.
- **Password locks**: If your PDFs are password-locked, you must remove the lock before submission.
- **Text height**: The minimum height of the text to be extracted is 12 pixels for a 1024 x 768-pixel image. This dimension corresponds to about 8-point text at 150 dots per inch.
- **Custom model training**: The maximum number of pages for training data is 500 for the custom template model and 50,000 for the custom neural model.
- **Custom extraction model training**: The total size of training data is 50 MB for template model and 1 GB for the neural model.
- **Custom classification model training**: The total size of training data is 1 GB with a maximum of 10,000 pages. For 2024-11-30 (GA), the total size of training data is 2 GB with a maximum of 10,000 pages.
- **Office file types (DOCX, XLSX, PPTX)**: The maximum string length limit is 8 million

characters.

Supported languages and locales

For a complete list of supported languages, *see* our [prebuilt model language support](../language-support/prebuilt) page.

Field extractions

For supported document extraction fields, *see* the [**bank statement model schema**](https://github.com/Azure-Samples/document-intelligence-code-samples/blob/main/schema/2024-11-30-ga/bank-statement.md) page in our GitHub sample repository.

Supported locales

The **prebuilt-bankStatement.us** version 2027-11-30 supports the **en-us** locale.

4. assistant (2026-05-31T04:18:49.454Z)

I have the full page content. The key facts are clear. Let me also note the page metadata for the publish/update date. The content explicitly indicates US-only support, cloud-based via Azure, and important constraints. Let me extract the falsifiable claims.

The page is Microsoft's official product documentation (primary source for the product's capabilities). Note one detail: there's an apparent typo in the doc ("version 2027-11-30") but the model is GA version 2024-11-30 per the feature table. I'll quote the locale claim accurately.

Key relevant facts for the research question:
1. Model only supports **US bank statements** / **en-us** locale ? directly relevant since muneem targets Indian banks.
2. **Password-locked PDFs must be unlocked before submission** ? relevant operational constraint.
3. Cloud-based Azure service (requires Azure subscription/endpoint) ? no on-prem/container mention on this page; relevant to the local-first hard constraint.
4. Extracts structured fields (account number, transaction details, fees) as JSON ? relevant to the table-parsing use case.
5. Page limits / file formats / model version ? supporting.

The Azure prebuilt bank statement model only supports the en-us locale; it does not support Indian bank statements (HDFC/Axis/ICICI/AU). It is a cloud Azure service with no on-prem/container option documented here, so it fails muneem's local-first hard constraint and would be an approval-gated cloud fallback that, in any case, isn't trained on Indian statement layouts.

5. user (2026-05-31T04:18:58.049Z)

Structured output provided successfully

6. assistant (2026-05-31T04:19:06.835Z)

Extracted the key claims from Microsoft's official Azure Document Intelligence bank statement documentation.

Summary of findings relevant to the muneem research question:

The Azure prebuilt bank statement model is a **disqualifying mismatch on two axes** for muneem:

1. **US-only / en-us locale** ? The model is explicitly trained for and supports only US bank statements. It is not trained on Indian bank layouts (HDFC, Axis, ICICI, AU Small Finance), which are muneem's actual targets. This alone makes it largely unusable even as a fallback.

2. **Cloud-only, fails the local-first hard constraint** ? It requires an Azure subscription and

a deployed cloud resource (key + endpoint). This page documents no on-premises/container deployment for this prebuilt model, so statement contents would have to leave the machine. That violates muneem's HARD CONSTRAINT #1 (100% local for sensitive statement data) and would make it strictly an approval-gated cloud fallback.

Operational note worth carrying into the report: the service **cannot ingest password-locked PDFs** (must be unlocked first) ? consistent with muneem's existing pypdf unlocking step. It does return structured JSON (account number, transaction details, fees, check tables as of V4.0 GA), accepts PDF up to 2,000 pages, 500 MB on paid tier.

Source quality: **primary** (Microsoft's official product documentation). Published 2025-11-18, last updated 2026-01-23.

One minor source artifact: the locale line reads "version 2027-11-30" which appears to be a typo in the doc ? the feature table lists the GA model as version 2024-11-30. I quoted it verbatim rather than silently correcting it.